

Folder server

26 printable files

(file list disabled)

server\.env

```
1 MONGO_URI=mongodb://localhost:27017/ecommerce
2 SECRET_KEY=my-secret-is-here-with-you-so-what-you-gonna-say
3
4 CLOUDINARY_CLOUD_NAME=daucssq7
5 CLOUDINARY_API_KEY=384892684148627
6 CLOUDINARY_API_SECRET=P5ftPXeG4u4AKNpM631p4cJ-3Xs
7
8 STRIPE_SECRET_KEY=sk_test_51SK1rj7c9l4eXzwynAj2xKf1YIjtKEedFU88ZQiRl22j7ZpAqwVt9pdpnvTMEqIY98TnYx1THuNJ2fg3JEJf6Gjj004HQkasQv
9 STRIPE_PUBLISHABLE_KEY=pk_test_51SK1rj7c9l4eXzwyuRAHQwBpZiRScpUQ9CBaXb8wKtwz1BdKLM4Ycg008o31ATacIsGDbNykQKJLzXP8iKrGF8Qu00XEYLl6tc
10 STRIPE_WEBHOOK_SECRET=whsec_251c51d70679fdbaa2d766dd17dfd47b2cbbfb528d3e63e50a19304cb1eb1fbd
11 CURRENCY=usd
```

server\config\db.js

```
1 import mongoose from "mongoose";
2 import dotenv from "dotenv";
3 dotenv.config();
4
5 const connectDB = async () => {
6   try {
7     await mongoose.connect(process.env.MONGO_URI, {
8       useNewUrlParser: true,
9       useUnifiedTopology: true,
10     });
11     console.log("MongoDB connected successfully");
12   } catch (error) {
13     console.error("Database connection failed:", error.message);
14     process.exit(1);
15   }
16 };
17
18 export default connectDB;
19
```

server\controllers\auth.js

```
1  import bcrypt from "bcrypt";
2  import jwt from "jsonwebtoken";
3  import User from "../models/User.js";
4
5  export const register = async (req, res) => {
6    try {
7      const { firstName, lastName, email, password, role } = req.body;
8
9      if (!firstName || !lastName || !email || !password)
10        return res
11          .status(400)
12          .json({ success: false, message: "All fields are required" });
13
14      const existingUser = await User.findOne({ email });
15      if (existingUser)
16        return res
17          .status(400)
18          .json({ success: false, message: "Email already registered" });
19
20      const hashedPassword = await bcrypt.hash(password, 10);
21      const newUser = await User.create({
22        firstName,
23        lastName,
24        email,
25        password: hashedPassword,
26        role: role || "customer",
27      });
28
29      const userObj = newUser.toObject();
30      delete userObj.password;
31
32      const token = jwt.sign(
33        { id: newUser._id, email: newUser.email, role: newUser.role },
34        process.env.SECRET_KEY,
35        { expiresIn: "7d" }
36      );
37
38      res.status(201).json({
39        success: true,
40        token,
```

```
41     user: userObj,  
42     message: "User registered successfully",  
43   });  
44   } catch (error) {  
45     res.status(500).json({ success: false, message: error.message });  
46   }  
47 };  
48  
49 export const login = async (req, res) => {  
50   try {  
51     const { email, password } = req.body;  
52     if (!email || !password)  
53       return res  
54         .status(400)  
55         .json({ success: false, message: "Email and password are required" });  
56  
57     const user = await User.findOne({ email }).select("+password");  
58     if (!user)  
59       return res  
60         .status(400)  
61         .json({ success: false, message: "Invalid credentials" });  
62  
63     const isMatch = await bcrypt.compare(password, user.password);  
64     if (!isMatch)  
65       return res  
66         .status(400)  
67         .json({ success: false, message: "Invalid credentials" });  
68  
69     const token = jwt.sign(  
70       { id: user._id, email: user.email, role: user.role },  
71       process.env.SECRET_KEY,  
72       { expiresIn: "7d" }  
73     );  
74  
75     const userObj = user.toObject();  
76     delete userObj.password;  
77  
78     res  
79       .status(200)  
80       .json({  
81         success: true,  
82         token,
```

```

83     user: userObj,
84     message: "Login successful",
85   });
86 } catch (error) {
87   res.status(500).json({ success: false, message: error.message });
88 }
89 };
90
91 export const logout = async (req, res) => {
92   try {
93     if (req.user && req.user.id) {
94       const user = await User.findById(req.user.id);
95       if (user) {
96         user.lastLogout = new Date();
97         await user.save();
98       }
99     }
100
101
102     res.setHeader("Authorization", "");
103     res.clearCookie("auth_token");
104     res.clearCookie("refresh_token");
105
106     res.status(200).json({
107       success: true,
108       message: "Logout successful",
109     });
110   } catch (error) {
111     console.error("Logout error:", error);
112
113     res.status(200).json({
114       success: true,
115       message: "Logout completed",
116     });
117   }
118 };
119
120 export const getMe = async (req, res) => {
121   try {
122     const user = await User.findById(req.user.id);
123
124     if (!user) {

```

```

125     return res.status(400).json({
126         success:false,
127         message:"User is not found"
128     })
129 };
130
131 res.status(200).json({
132     success:true,
133     user:{
134         id:user._id,
135         firstName:user.firstName,
136         lastName:user.lastName,
137         email:user.email,
138         role:user.role,
139     }
140 })
141
142 } catch (error) {
143     console.error("Get me error:", error);
144     res.status(500).json({
145         success: false,
146         message: "Server error",
147     });
148 }
149 }

```

server\controllers\cartController.js

```

1  import Cart from "../models/Cart.js";
2
3  export const addToCart = async (req, res) => {
4      try {
5          const userId = req.user._id;
6          const { productId, quantity = 1, price } = req.body;
7
8          if (!productId || !price) {
9              return res
10                 .status(400)
11                 .json({ success: false, message: "productId and price are required" });
12          }
13
14          let cart = await Cart.findOne({ user: userId });

```

```

15   if (!cart) {
16       cart = new Cart({
17           user: userId,
18           items: [{ product: productId, quantity, price }],
19           totalPrice: quantity * price,
20       });
21   } else {
22       const itemIndex = cart.items.findIndex(
23           (item) => item.product.toString() === productId
24       );
25       if (itemIndex > -1) {
26           cart.items[itemIndex].quantity += quantity;
27       } else {
28           cart.items.push({ product: productId, quantity, price });
29       }
30       cart.totalPrice = cart.items.reduce(
31           (total, item) => total + item.quantity * item.price,
32           0
33       );
34   }
35
36   await cart.save();
37   res.status(200).json({ success: true, cart });
38 } catch (error) {
39   res.status(500).json({ success: false, message: error.message });
40 }
41 };
42
43 export const getUserCart = async (req, res) => {
44   try {
45       const userId = req.user._id;
46       const cart = await Cart.findOne({ user: userId }).populate("items.product");
47       if (!cart)
48           return res
49               .status(404)
50               .json({ success: false, message: "Cart not found" });
51       res.status(200).json({ success: true, cart });
52   } catch (error) {
53       res.status(500).json({ success: false, message: error.message });
54   }
55 };
56

```

```

57 export const updateItemQuantity = async (req, res) => {
58   try {
59     const userId = req.user._id;
60     const { productId, quantity } = req.body;
61     const cart = await Cart.findOne({ user: userId });
62     if (!cart)
63       return res
64         .status(404)
65         .json({ success: false, message: "Cart not found" });
66
67     const item = cart.items.find((i) => i.product.toString() === productId);
68     if (!item)
69       return res
70         .status(404)
71         .json({ success: false, message: "Item not found in cart" });
72
73     item.quantity = quantity;
74     cart.totalPrice = cart.items.reduce(
75       (total, it) => total + it.quantity * it.price,
76       0
77     );
78     await cart.save();
79     res.status(200).json({ success: true, cart });
80   } catch (error) {
81     res.status(500).json({ success: false, message: error.message });
82   }
83 };
84
85 export const removeCartItem = async (req, res) => {
86   try {
87     const userId = req.user._id;
88     const { productId } = req.params;
89
90     const cart = await Cart.findOne({ user: userId });
91     if (!cart)
92       return res
93         .status(404)
94         .json({ success: false, message: "Cart not found" });
95
96     cart.items = cart.items.filter(
97       (item) => item.product.toString() !== productId
98     );

```

```

99     cart.totalPrice = cart.items.reduce(
100         (total, item) => total + item.quantity * item.price,
101         0
102     );
103
104     await cart.save();
105     res.status(200).json({ success: true, message: "Item removed", cart });
106 } catch (error) {
107     res.status(500).json({ success: false, message: error.message });
108 }
109 };
110

```

server\controllers\categoryController.js

```

1  import Category from "../models/Category.js";
2  import Product from "../models/Product.js";
3
4  export const createCategory = async (req, res) => {
5      try {
6          const { name, description } = req.body;
7
8          if (!name) {
9              return res.status(400).json({
10                  success: false,
11                  message: "name is required",
12              });
13          }
14
15          if (req.user.role !== "admin") {
16              return res.status(403).json({
17                  success: false,
18                  message: "Access denied. Only admin can create categories.",
19              });
20          }
21
22          const existingCategory = await Category.findOne({ name });
23          if (existingCategory) {
24              return res.status(400).json({
25                  success: false,
26                  message: "Category already exists",
27              });

```

```
28     }
29
30     const category = await Category.create({
31       name,
32       description,
33     });
34
35     res.status(201).json({
36       success: true,
37       message: "Category created successfully",
38       category,
39     });
40   } catch (error) {
41     res.status(500).json({
42       success: false,
43       message: error.message,
44     });
45   }
46 };
47
48 export const getAllCategories = async (req, res) => {
49   try {
50     const categories = await Category.find();
51     if (!categories || categories.length === 0) {
52       return res.status(404).json({
53         success: false,
54         message: "No categories found",
55       });
56     }
57
58     res.status(200).json({
59       count: categories.length,
60       success: true,
61       categories,
62     });
63   } catch (error) {
64     res.status(500).json({
65       success: false,
66       message: error.message,
67     });
68   }
69 };
```

```
70
71 export const updateCategory = async (req, res) => {
72   try {
73     const { name, description } = req.body;
74
75     if (req.user.role !== "admin") {
76       return res.status(400).json({
77         success: false,
78         message: "Access denied. Only admin can update categories.",
79       });
80     }
81
82     const category = await Category.findById(req.params.id);
83     if (!category) {
84       return res.status(400).json({
85         success: false,
86         message: "category not found",
87       });
88     }
89
90     const updatedCategory = await Category.findByIdAndUpdate(
91       req.params.id,
92       {
93         name,
94         description,
95       },
96       {
97         new: true,
98         runValidators: true,
99       }
100     );
101
102     res.status(200).json({
103       success: true,
104       message: "Category updated successfully",
105       category: updatedCategory,
106     });
107   } catch (error) {
108     res.status(500).json({
109       success: false,
110       message: error.message,
111     });
112   }
113 }
```

```

112 }
113 };
114
115 export const deleteCategory = async (req, res) => {
116   try {
117     if (req.user.role !== "admin") {
118       return res.status(400).json({
119         success: false,
120         message: "Access denied. Only admin can delete categories.",
121       });
122     }
123
124     const category = await Category.findById(req.params.id);
125     if (!category) {
126       return res.status(404).json({
127         success: false,
128         message: "Category not found",
129       });
130     }
131
132     await Category.findByIdAndDelete(req.params.id);
133
134     res.status(200).json({
135       success: true,
136       message: "Category deleted successfully!",
137     });
138   } catch (error) {
139     res.status(500).json({
140       success: false,
141       message: error.message,
142     });
143   }
144 };
145

```

server\controllers\orderController.js

```

1 import Order from "../models/Order.js";
2 import Cart from "../models/Cart.js";
3
4 export const createOrder = async (req, res) => {
5   try {

```

```
6   const userId = req.user._id;
7   const { shippingAddress } = req.body;
8
9   const cart = await Cart.findOne({ user: userId }).populate("items.product");
10  if (!cart || cart.items.length === 0) {
11    return res
12      .status(400)
13      .json({ success: false, message: "Cart is empty or not found" });
14  }
15
16  const totalPrice = cart.items.reduce(
17    (acc, item) => acc + item.quantity * item.price,
18    0
19  );
20
21  const order = await Order.create({
22    user: userId,
23    orderItems: cart.items.map((item) => ({
24      product: item.product._id,
25      quantity: item.quantity,
26      price: item.price,
27    })),
28    shippingAddress,
29    totalPrice,
30  });
31
32  await Cart.deleteOne({ user: userId });
33
34  res
35    .status(201)
36    .json({ success: true, message: "Order placed successfully", order });
37  } catch (error) {
38    res.status(500).json({ success: false, message: error.message });
39  }
40 };
41
42 export const getUserOrders = async (req, res) => {
43   try {
44     const userId = req.user._id;
45     const orders = await Order.find({ user: userId }).populate(
46       "orderItems.product"
47     );
```

```
48     res.status(200).json({ success: true, orders });
49   } catch (error) {
50     res.status(500).json({ success: false, message: error.message });
51   }
52 };
53
54
55 export const updateOrderStatus = async (req, res) => {
56   try {
57     const { status } = req.body;
58     const order = await Order.findById(req.params.id);
59     if (!order)
60       return res
61         .status(404)
62         .json({ success: false, message: "Order not found" });
63
64     order.status = status;
65     await order.save();
66
67     res
68       .status(200)
69       .json({
70         success: true,
71         count: order.length,
72         message: `Order status updated to ${status}`,
73         order,
74       });
75   } catch (error) {
76     res.status(500).json({ success: false, message: error.message });
77   }
78 };
79
80 export const deleteOrder = async (req, res) => {
81   try {
82     const order = await Order.findById(req.params.id);
83     if (!order)
84       return res
85         .status(404)
86         .json({ success: false, message: "Order not found" });
87
88     await order.deleteOne();
89     res
```

```
90     .status(200)
91     .json({ success: true, message: "Order deleted successfully" });
92   } catch (error) {
93     res.status(500).json({ success: false, message: error.message });
94   }
95 };
96
```

server\controllers\paymentController.js

```
1  import Stripe from "stripe";
2  import Order from "../models/Order.js";
3  import Cart from "../models/Cart.js";
4
5  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
6
7  export const createPaymentIntent = async (req, res) => {
8    try {
9      const userId = req.user?.id || req.user?._id;
10     const { orderId } = req.body;
11
12     if (!userId && !orderId) {
13       return res
14         .status(400)
15         .json({ success: false, message: "userId or orderId required" });
16     }
17
18     let amountCents;
19     let order;
20
21     if (orderId) {
22       order = await Order.findById(orderId).populate("orderItems.product");
23       if (!order)
24         return res
25           .status(404)
26           .json({ success: false, message: "Order not found" });
27       amountCents = Math.round(order.totalPrice * 100);
28     } else {
29       const cart = await Cart.findOne({ user: userId }).populate(
30         "items.product"
31       );
32       if (!cart || !cart.items.length)
```

```

33     return res
34     .status(400)
35     .json({ success: false, message: "Cart not found or empty" });
36     amountCents = Math.round(cart.totalPrice * 100);
37 }
38
39 const paymentIntent = await stripe.paymentIntents.create({
40     amount: amountCents,
41     currency: process.env.CURRENCY || "usd",
42     metadata: { orderId: orderId || `fromCart:${userId}` },
43     automatic_payment_methods: { enabled: true },
44 });
45
46 res
47     .status(200)
48     .json({ success: true, clientSecret: paymentIntent.client_secret });
49 } catch (err) {
50     console.error("createPaymentIntent:", err);
51     res.status(500).json({ success: false, message: err.message });
52 }
53 };
54
55 export const markOrderAsPaid = async (req, res) => {
56     try {
57         const order = await Order.findById(req.params.id);
58         if (!order)
59             return res
60                 .status(404)
61                 .json({ success: false, message: "Order not found" });
62
63         order.isPaid = true;
64         order.paidAt = Date.now();
65         order.paymentMethod = req.body.paymentMethod || "Card";
66
67         order.paymentResult = {
68             id: req.body.paymentId || "SIMULATED_TXN_" + Date.now(),
69             status: "Completed",
70             update_time: new Date().toISOString(),
71             email_address: req.body.email || "user@example.com",
72         };
73
74         await order.save();

```

```

75     res
76     .status(200)
77     .json({
78         success: true,
79         message: "Payment confirmed successfully",
80         order,
81     });
82 } catch (error) {
83     res.status(500).json({ success: false, message: error.message });
84 }
85 };
86

```

server\controllers\productController.js

```

1  import { v2 as cloudinary } from "cloudinary";
2  import Product from "../models/Product.js";
3  import Category from "../models/Category.js";
4
5
6  export const createProduct = async (req, res) => {
7      try {
8          const { name, description, price, category, stock } = req.body;
9
10         if (!name || !description || !price || !category || !stock) {
11             return res.status(400).json({
12                 success: false,
13                 message: "All fields are required",
14             });
15         }
16
17         const images =
18             req.files?.length > 0
19             ? req.files.map((file) => ({
20                 url: file.path,
21                 filename: file.filename,
22             }))
23             : [];
24
25         const productData = {
26             name,
27             description,

```

```
28     price,
29     category,
30     stock,
31   };
32
33   if (images.length > 0) productData.image = images;
34
35   const product = await Product.create(productData);
36
37   res.status(201).json({
38     success: true,
39     message: "Product created successfully",
40     product,
41   });
42 } catch (error) {
43   res.status(400).json({
44     success: false,
45     message: error.message,
46   });
47 }
48 };
49
50 export const getAllProducts = async (req, res) => {
51   try {
52     const { category, keyword } = req.query;
53     let filter = {};
54
55     if (category) {
56       const cat = await Category.findOne({
57         name: { $regex: category, $options: "i" },
58       });
59       if (cat) filter.category = cat._id;
60     }
61
62     if (keyword) {
63       filter.name = { $regex: keyword, $options: "i" };
64     }
65
66     const products = await Product.find(filter).populate("category", "name");
67     res.json({ count: products.length, products });
68   } catch (err) {
69     res.status(500).json({ success: false, message: err.message });
70   }
71 }
```

```
70 }
71 };
72
73 export const getProductById = async (req, res) => {
74   try {
75     const product = await Product.findById(req.params.id).populate(
76       "category",
77       "name"
78     );
79     if (!product)
80       return res
81         .status(404)
82         .json({ success: false, message: "Product not found" });
83     res.status(200).json({ success: true, product });
84   } catch (error) {
85     res.status(400).json({ success: false, message: error.message });
86   }
87 };
88
89 export const updateProduct = async (req, res) => {
90   try {
91     const { name, description, price, category, stock } = req.body;
92
93     const product = await Product.findById(req.params.id);
94     if (!product)
95       return res
96         .status(404)
97         .json({ success: false, message: "Product not found" });
98
99     if (req.user.role !== "admin")
100       return res
101         .status(403)
102         .json({ success: false, message: "Not authorized" });
103
104
105     if (name) product.name = name;
106     if (description) product.description = description;
107     if (price) product.price = price;
108     if (category) product.category = category;
109     if (stock !== undefined) product.stock = stock;
110
111
```

```
112   if (req.files && req.files.length > 0) {
113     for (let img of product.image) {
114       await cloudinary.uploader.destroy(img.filename);
115     }
116
117     product.image = req.files.map((file) => ({
118       url: file.path,
119       filename: file.filename,
120     }));
121   }
122
123   await product.save();
124
125   res.status(200).json({
126     success: true,
127     message: "Product updated successfully",
128     product,
129   });
130 } catch (error) {
131   res.status(400).json({ success: false, message: error.message });
132 }
133 };
134
135
136 export const deleteProduct = async (req, res) => {
137   try {
138     const product = await Product.findById(req.params.id);
139     if (!product)
140       return res
141         .status(404)
142         .json({ success: false, message: "Product not found" });
143
144     if (req.user.role !== "admin")
145       return res
146         .status(403)
147         .json({ success: false, message: "Not authorized to delete products" });
148
149     await Product.findByIdAndDelete(req.params.id);
150     res
151       .status(200)
152       .json({ success: true, message: "Product deleted successfully" });
153   } catch (error) {
```

```
154     res.status(400).json({ success: false, message: error.message });
155   }
156 };
157
```

server\controllers\webhookController.js

```
1  import Stripe from "stripe";
2  import mongoose from "mongoose";
3  import Order from "../models/Order.js";
4  import Product from "../models/Product.js";
5
6  const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
7
8  export const stripeWebhook = async (req, res) => {
9    const sig = req.headers["stripe-signature"];
10    const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET;
11
12    let event;
13    try {
14      event = stripe.webhooks.constructEvent(req.rawBody, sig, webhookSecret);
15    } catch (err) {
16      console.error("Webhook signature verification failed:", err.message);
17      return res.status(400).send(`Webhook Error: ${err.message}`);
18    }
19
20    try {
21      if (event.type === "payment_intent.succeeded") {
22        const paymentIntent = event.data.object;
23        const metadataOrderId = paymentIntent.metadata?.orderId;
24        let orderId = metadataOrderId;
25
26        if (metadataOrderId?.startsWith("fromCart:")) {
27          console.log("Payment for cart (no orderId):", metadataOrderId);
28        }
29
30        if (orderId) {
31          const session = await mongoose.startSession();
32          try {
33            session.startTransaction();
34
35            const order = await Order.findById(orderId).session(session);
```

```
36     if (!order) {
37         console.warn("Order not found for orderId:", orderId);
38         await session.commitTransaction();
39     } else if (!order.isPaid) {
40         order.isPaid = true;
41         order.paidAt = new Date();
42         order.paymentMethod = "Card";
43         order.paymentResult = {
44             id: paymentIntent.id,
45             status: paymentIntent.status,
46             update_time: new Date().toISOString(),
47             email_address: paymentIntent.receipt_email || "",
48         };
49
50         for (const item of order.orderItems) {
51             const updateResult = await Product.updateOne(
52                 { _id: item.product, stock: { $gte: item.quantity } },
53                 { $inc: { stock: -item.quantity } }
54             ).session(session);
55
56             if (updateResult.matchedCount === 0) {
57                 throw new Error(
58                     `Insufficient stock for product ${item.product}`
59                 );
60             }
61         }
62
63         await order.save({ session });
64         await session.commitTransaction();
65         console.log(`Order ${orderId} marked paid and stock updated.`);
66     } else {
67         await session.commitTransaction();
68         console.log(`Order ${orderId} already marked as paid.`);
69     }
70 } catch (txErr) {
71     await session.abortTransaction();
72     console.error("Transaction error in webhook:", txErr);
73 } finally {
74     session.endSession();
75 }
76 }
77 } else {
```

```
78     console.log("Unhandled stripe event:", event.type);
79 }
80
81     return res.json({ received: true });
82 } catch (err) {
83     console.error("Webhook processing error:", err);
84     return res.status(500).send();
85 }
86 };
```

server\middleware\auth.js

```
1 import jwt from "jsonwebtoken";
2 import User from "../models/User.js";
3
4 export const verifyUser = async (req, res, next) => {
5     try {
6         let token = req.header("Authorization");
7         if (!token)
8             return res.status(401).json({ success: false, message: "Access Denied" });
9
10        if (token.startsWith("Bearer ")) {
11            token = token.slice(7).trim();
12        }
13
14        const decoded = jwt.verify(token, process.env.SECRET_KEY);
15        const user = await User.findById(decoded.id).select("-password");
16        if (!user) {
17            return res
18                .status(404)
19                .json({ success: false, message: "User not found" });
20        }
21
22        req.user = user;
23        next();
24    } catch (error) {
25        if (error.name === "TokenExpiredError") {
26            return res.status(401).json({ success: false, message: "Token expired" });
27        }
28        if (error.name === "JsonWebTokenError") {
29            return res.status(401).json({ success: false, message: "Invalid token" });
30        }
31    }
32 }
```

```

31     return res
32     .status(401)
33     .json({ success: false, message: "Authentication failed" });
34 }
35 };
36
37 export const authorize = (...roles) => {
38     return (req, res, next) => {
39         if (!req.user) {
40             return res.status(401).json({
41                 success: false,
42                 message: "Access denied",
43             });
44         }
45         if (!roles.includes(req.user.role)) {
46             return res.status(403).json({
47                 success: false,
48                 message: `User role ${req.user.role} is not authorized`,
49             });
50         }
51         next();
52     };
53 };
54

```

server\middleware\upload.js

```

1  import multer from "multer";
2  import { v2 as cloudinary } from "cloudinary";
3  import { CloudinaryStorage } from "multer-storage-cloudinary";
4
5  cloudinary.config({
6      cloud_name: process.env.CLOUDINARY_CLOUD_NAME,
7      api_key: process.env.CLOUDINARY_API_KEY,
8      api_secret: process.env.CLOUDINARY_API_SECRET,
9  });
10
11  const storage = new CloudinaryStorage({
12      cloudinary,
13      params: {
14          folder: "ecommerce",
15          allowed_formats: ["jpg", "jpeg", "png", "webp"],

```

```

16     transformation: [{ width: 800, height: 800, crop: "limit" }],
17   },
18 });
19
20 const upload = multer({
21   storage,
22   limits: {
23     fileSize: 5 * 1024 * 1024,
24   },
25   fileFilter: (req, file, cb) => {
26     if (file.mimetype.startsWith("image/")) {
27       cb(null, true);
28     } else {
29       cb(new Error("Only image files are allowed"), false);
30     }
31   },
32 });
33
34 export default upload;
35

```

server\models\Cart.js

```

1  import mongoose from "mongoose";
2
3  const cartSchema = new mongoose.Schema(
4    {
5      user: {
6        type: mongoose.Schema.Types.ObjectId,
7        ref: "User",
8        required: true,
9      },
10     items: [
11       {
12         product: {
13           type: mongoose.Schema.Types.ObjectId,
14           ref: "Product",
15           required: true,
16         },
17         quantity: {
18           type: Number,
19           required: true,

```

```

20         min: 1,
21         default: 1,
22     },
23     price: {
24         type: Number,
25         required: true,
26     },
27 },
28 ],
29 totalPrice: {
30     type: Number,
31     required: true,
32     default: 0,
33 },
34 },
35 { timestamps: true }
36 );
37
38 export default mongoose.model("Cart", cartSchema);
39

```

server\models\Category.js

```

1  import mongoose from "mongoose";
2
3  const categorySchema = new mongoose.Schema(
4      {
5          name: {
6              type: String,
7              required: true,
8              unique: true,
9              trim: true,
10         },
11         description: {
12             type: String,
13             default: "",
14         },
15     },
16     { timestamps: true }
17 );
18
19 const Category = mongoose.model("Category", categorySchema);

```

```
20 export default Category;
```

```
21
```

server\models\Order.js

```
1  import mongoose from "mongoose";
2
3  const orderSchema = new mongoose.Schema(
4    {
5      user: {
6        type: mongoose.Schema.Types.ObjectId,
7        ref: "User",
8        required: true,
9      },
10     orderItems: [
11       {
12         product: {
13           type: mongoose.Schema.Types.ObjectId,
14           ref: "Product",
15           required: true,
16         },
17         quantity: {
18           type: Number,
19           required: true,
20           min: 1,
21         },
22         price: {
23           type: Number,
24           required: true,
25         },
26       },
27     ],
28     shippingAddress: {
29       address: { type: String, required: true },
30       city: { type: String, required: true },
31       postalCode: { type: String },
32       country: { type: String, required: true },
33     },
34     totalPrice: {
35       type: Number,
36       required: true,
37       default: 0,
```

```

38     },
39     paymentMethod: {
40       type: String,
41       enum: ["Cash on Delivery", "Card", "PayPal"],
42       default: "Card",
43     },
44     paymentResult: {
45       id: String,
46       status: String,
47       update_time: String,
48       email_address: String,
49     },
50     isPaid: {
51       type: Boolean,
52       default: false,
53     },
54     paidAt: Date,
55     status: {
56       type: String,
57       enum: ["Pending", "Processing", "Shipped", "Delivered", "Cancelled"],
58       default: "Pending",
59     },
60   },
61   { timestamps: true }
62 );
63
64 export default mongoose.model("Order", orderSchema);
65

```

server\models\Product.js

```

1  import mongoose from 'mongoose';
2
3  const productSchema = new mongoose.Schema({
4    name: {
5      type: String,
6      required: true
7    },
8    description: {
9      type: String,
10     required: true
11   },

```

```

12 price: {
13     type: Number,
14     required: true
15 },
16 stock: {
17     type: Number,
18     required: true
19 },
20 image: [{
21     url: String,
22     filename: String,
23
24 }],
25 category: {
26     type: mongoose.Schema.Types.ObjectId,
27     ref: "Category",
28     required: true
29 }
30 })
31 const Product = mongoose.model("Product", productSchema);
32 export default Product;
33

```

server\models\Review.js

```

1 import mongoose from "mongoose";
2
3 const reviewSchema = new mongoose.Schema(
4     {
5         userId: {
6             type: mongoose.Schema.Types.ObjectId,
7             ref: "User",
8             required: true,
9         },
10        productId: {
11            type: mongoose.Schema.Types.ObjectId,
12            ref: "Product",
13            required: true,
14        },
15        rating: { type: Number, required: true, min: 1, max: 5 },
16        comment: { type: String },
17        images: [{ type: String }],

```

```
18   isVerifiedPurchase: { type: Boolean, default: false },
19   },
20   {
21     timestamps: true,
22   }
23 );
24
25 const Review = mongoose.model("Review", reviewSchema);
26 export default Review;
27
```

server\models\User.js

```
1 import mongoose from "mongoose";
2 const userSchema = new mongoose.Schema(
3   {
4     firstName: {
5       type: String,
6       required: true,
7     },
8     lastName: {
9       type: String,
10      required: true,
11    },
12    email: {
13      type: String,
14      required: true,
15      unique: true,
16    },
17    password: {
18      type: String,
19      required: true,
20      select: false,
21      min: 6,
22    },
23    role: {
24      type: String,
25      enum: ["customer", "admin"],
26      default: "customer",
27    },
28  },
29  { timestamps: true }
```

```

30 );
31
32 const User = mongoose.model("User", userSchema);
33 export default User;
34

```

server\package.json

```

1  {
2    "name": "server",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1",
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "type": "module",
14   "dependencies": {
15     "bcrypt": "^6.0.0",
16     "body-parser": "^2.2.0",
17     "cloudinary": "^2.7.0",
18     "dotenv": "^17.2.3",
19     "express": "^5.1.0",
20     "jsonwebtoken": "^9.0.2",
21     "mongoose": "^8.19.1",
22     "multer": "^2.0.2",
23     "multer-storage-cloudinary": "^4.0.0",
24     "nodemon": "^3.1.10",
25     "stripe": "^19.1.0"
26   }
27 }
28

```

server\routes\auth.js

```

1  import express from 'express';
2  import { verifyUser, authorize } from '../middleware/auth.js';
3  import { login, logout, register, getMe } from '../controllers/auth.js';
4  const router = express.Router();

```

```
5 router.post(' /register',register);
6 router.post(' /login',login);
7 router.get("/me",verifyUser,getMe)
8 router.post(' /logout',verifyUser,logout);
9
10 export default router;
```

server\routes\cartRoutes.js

```
1 import express from "express";
2 import {
3   addToCart,
4   getUserCart,
5   removeCartItem,
6   updateItemQuantity,
7 } from "../controllers/cartController.js";
8 import { verifyUser } from "../middleware/auth.js";
9
10 const router = express.Router();
11
12 router.post("/", verifyUser, addToCart);
13 router.get("/:userId", verifyUser, getUserCart);
14 router.put("/:userId", verifyUser, updateItemQuantity);
15 router.delete("/:userId/:productId", verifyUser, removeCartItem);
16
17 export default router;
18
```

server\routes\categoryRoutes.js

```
1 import express from "express";
2 import {
3   createCategory,
4   deleteCategory,
5   getAllCategories,
6   updateCategory,
7 } from "../controllers/categoryController.js";
8 import { authorize, verifyUser } from "../middleware/auth.js";
9
10 const router = express.Router();
11
12 router.get("/", getAllCategories);
13 router.post("/", verifyUser, authorize("admin"), createCategory);
```

```
14 router.put("/:id", verifyUser, authorize("admin"), updateCategory);
15 router.delete("/:id", verifyUser, authorize("admin"), deleteCategory);
16
17 export default router;
18
```

server\routes\orderRoutes.js

```
1 import express from "express";
2 import {
3   createOrder,
4   getUserOrders,
5   updateOrderStatus,
6   deleteOrder,
7 } from "../controllers/orderController.js";
8 import { markOrderAsPaid } from "../controllers/paymentController.js";
9 import { authorize, verifyUser } from "../middleware/auth.js";
10
11 const router = express.Router();
12
13 router.post("/", verifyUser, createOrder);
14 router.get("/my-orders", verifyUser, getUserOrders);
15
16
17 router.put("/:id/status", verifyUser, authorize("admin"), updateOrderStatus);
18 router.delete("/:id", verifyUser, authorize("admin"), deleteOrder);
19
20 export default router;
21
```

server\routes\paymentRoutes.js

```
1 import express from "express";
2 import { createPaymentIntent, markOrderAsPaid } from "../controllers/paymentController.js";
3 import { authorize, verifyUser } from "../middleware/auth.js";
4
5 const router = express.Router();
6
7 router.post("/create-payment-intent", verifyUser, authorize("admin"), createPaymentIntent);
8
9 router.put("/order/:id/mark-paid", verifyUser, authorize("admin"), markOrderAsPaid);
10
11 export default router;
```

12

server\routes\productRoutes.js

```
1 import express from 'express'
2 import {getAllProducts,createProduct,getProductById,deleteProduct,updateProduct} from '../controllers/productController.js';
3
4 import { verifyUser,authorize } from '../middleware/auth.js';
5 import upload from "../middleware/upload.js";
6
7 const router = express.Router();
8
9 router.get('/', getAllProducts);
10 router.get('/:id', getProductById);
11
12 router.post('/', verifyUser, authorize('admin'), upload.array('image', 5), createProduct);
13 router.put('/:id', verifyUser, authorize('admin'),upload.array('image',5), updateProduct);
14 router.delete('/:id', verifyUser, authorize('admin'), deleteProduct);
15
16 export default router;
```

server\routes\webhookRoutes.js

```
1 import express from "express";
2 const router = express.Router();
3
4 router.post(
5   "/",
6   express.raw({ type: "application/json" }),
7   (req, res) => {
8     req.rawBody = req.body;
9     req.body = {};
10     import("../controllers/webhookController.js").then(module => {
11       module.stripeWebhook(req, res);
12     });
13   }
14 );
15
16 export default router;
```

server\server.js

```
1 import express from "express";
2 import dotenv from "dotenv";
```

```
3 import connectDB from "./config/db.js";
4 import authRoutes from "./routes/auth.js";
5 import productRoutes from "./routes/productRoutes.js";
6 import categoryRoutes from "./routes/categoryRoutes.js";
7 import cartRoutes from "./routes/cartRoutes.js";
8 import orderRoutes from "./routes/orderRoutes.js";
9 import paymentRoutes from "./routes/paymentRoutes.js";
10 import webhookRoutes from "./routes/webhookRoutes.js";
11
12 dotenv.config();
13 connectDB();
14
15 const app = express();
16
17 app.use("/webhook", webhookRoutes);
18
19 app.use(express.json());
20 app.use("/api/auth", authRoutes);
21 app.use("/api/products", productRoutes);
22 app.use("/api/categories", categoryRoutes);
23 app.use("/api/cart", cartRoutes);
24 app.use("/api/orders", orderRoutes);
25 app.use("/api/payments", paymentRoutes);
26
27 const PORT = process.env.PORT || 5000;
28 app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
29
```